Контейнеризация REST API и базы данных

От проверки соединения — к полноценному CRUD товаров через Nginx, Express API и PostgreSQL

Цель лекции

- Понимать архитектуру контейнеризированного CRUD-приложения.
- Создать таблицу products в PostgreSQL и выполнять параметризованные SQL-запросы.
- Реализовать маршруты GET, POST, PATCH, DELETE в Express.
- Организовать подключение через pg.Pool и Docker Compose.
- Проверять HTTP-коды, readiness и сохранность данных.

Повторение темы №7

- Compose объединяет nginx, api, db и обеспечивает встроенный DNS по именам сервисов.
- Nginx принимает запросы на порту хоста 8080 и перенаправляет их к api:3000.
- Express использует DB_HOST=db и DB_PORT=5432.
- PostgreSQL 17 сохраняет данные в именованном томе pgdata.
- Сейчас заменим демонстрационный /api/db-check на рабочий /api/products.

Учебные вопросы

№	Вопрос	Практическое применение
1	Архитектура CRUD и ответственность сервисов	Путь HTTP → API → SQL → JSON
2	Таблица PostgreSQL и SQL CRUD	products, SELECT/INSERT/UPDATE/DELETE
3	Express + pg, маршруты и валидация	Получение и изменение товаров
4	Dockerfile, Compose и готовность БД	Воспроизводимый запуск трёх сервисов
5	Проверка, диагностика и сохранность данных	curl, logs, psql, volume

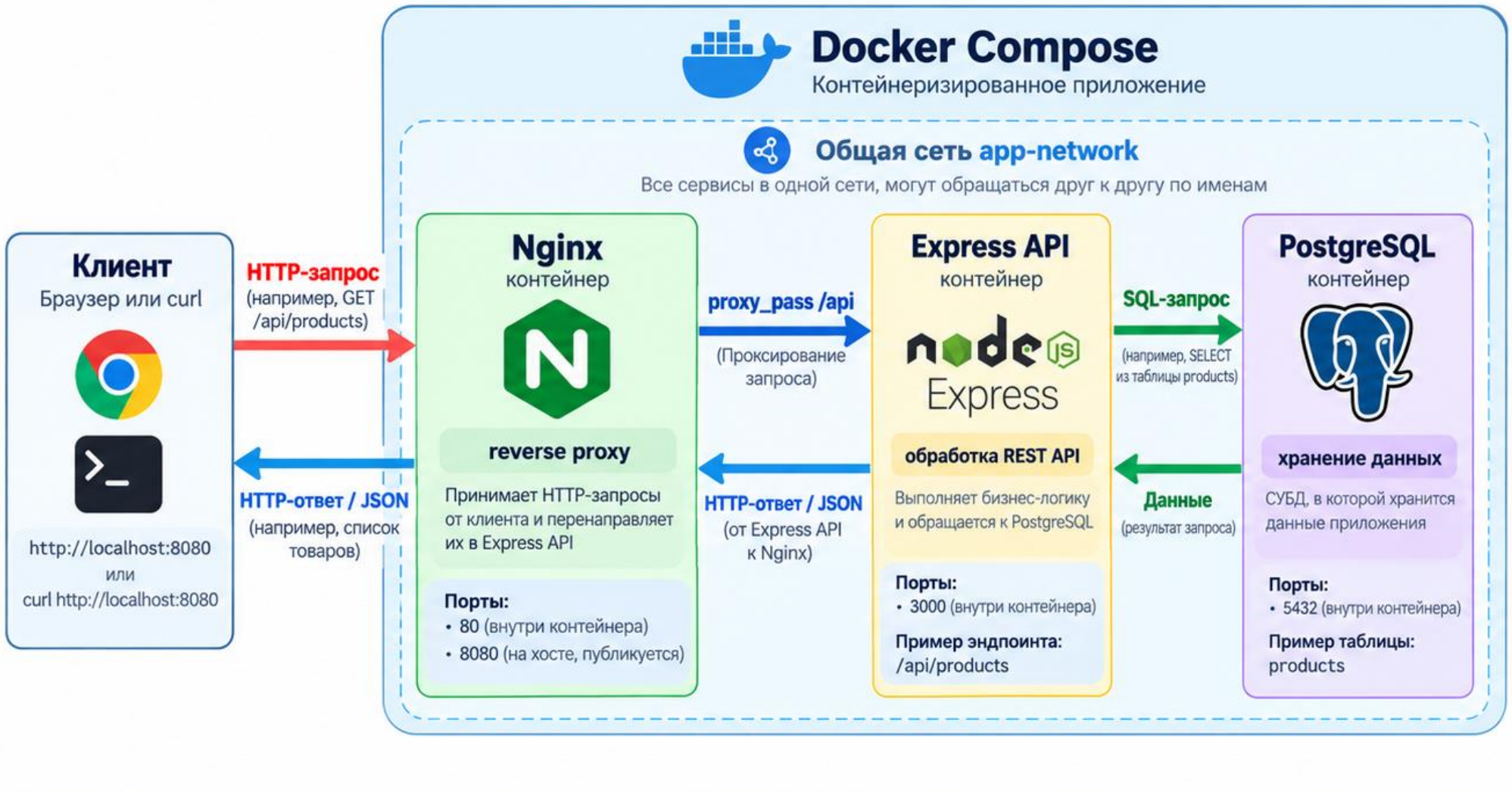
Архитектура контейнеризированного REST API

Как распределены обязанности nginx, api и db

Три контейнера — три ответственности

- nginx: обратный прокси, единая внешняя точка входа.
- api: маршрутизация REST, проверка входных данных, бизнес-логика.
- db: хранение таблиц, SQL и транзакции.
- Связь между сервисами осуществляется внутри сети Compose; на хост публикуется только Nginx.

Путь запроса Nginx → Express API → PostgreSQL



Этапы выполнения запроса

1. **Клиент обращается к http://localhost:8080**
   Например, открывает в браузере или делает запрос через curl.
2. **Nginx принимает запрос и перенаправляет его в Express API**
   Использует правило `proxy_pass /api`.
3. **Express выполняет логику и обращается к PostgreSQL**
   Например, получает список товаров из базы данных.
4. **PostgreSQL возвращает данные**
   Результат SQL-запроса (например, данные из таблицы `products`).
5. **Express формирует JSON-ответ**
   Преобразует данные в JSON.
6. **Nginx возвращает ответ клиенту**
   Отправляет JSON через HTTP-ответ.

Что важно понять

1. **Nginx — точка входа для клиента**
   Принимает внешние HTTP-запросы и проксирует их в Express API.
2. **Express API обрабатывает бизнес-логику**
   Реализует REST API, работает с данными и обращается к PostgreSQL.
3. **PostgreSQL хранит данные приложения**
   В базе данных хранится вся основная информация (например, таблица products).

Путь запроса GET /api/products

1	**Шаг 1**	1. Клиент обращается к http://localhost:8080/api/products.
2	**Шаг 2**	2. nginx выполняет proxy_pass на http://api:3000.
3	**Шаг 3**	3. Express Router передаёт запрос Controller.
4	**Шаг 4**	4. Service выполняет SQL SELECT через пул подключений pg.
5	**Шаг 5**	5. PostgreSQL возвращает строки; API формирует JSON и статус 200.
6	**Шаг 6**	6. Nginx пересылает HTTP-ответ клиенту.

Данные приложения и конфигурация

В образе API

- Исходный код сервера
- package.json и package-lock.json
- Установленные зависимости
- Команда запуска Node.js

Вне образа API

- Строки таблицы products в PostgreSQL
- Секреты и значения переменных окружения
- Постоянный Docker Volume
- Опубликованный порт и сеть Compose

REST как контракт между клиентом и сервером

- GET /api/products — получить список (200 OK).
- GET /api/products/:id — получить товар (200 или 404).
- POST /api/products — создать товар (201 Created).
- PATCH /api/products/:id — изменить поля (200 или 404).
- DELETE /api/products/:id — удалить товар (204 No Content или 404).

Таблица PostgreSQL и SQL CRUD

Как данные сохраняются независимо от контейнера API

Таблица products

```sql
CREATE TABLE IF NOT EXISTS products (
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  price NUMERIC(12,2) NOT NULL CHECK (price >= 0)
);
```

Что важно

- id генерируется PostgreSQL; name обязателен, price не может быть отрицательным.
- Таблица создаётся отдельно от файлов Express; pgdata обеспечивает сохранность физических данных БД.

Типы данных и ограничения

- INTEGER GENERATED ALWAYS AS IDENTITY: идентификатор назначает СУБД.
- VARCHAR(100) NOT NULL: непустое на уровне NULL, пустую строку дополнительно проверяет API.
- NUMERIC(12,2): точное десятичное представление стоимости; pg обычно возвращает NUMERIC как строку.
- CHECK (price >= 0): дополнительная защита от отрицательной цены.
- Ограничения БД дополняют, но не отменяют валидацию HTTP-входа.

Чтение товаров

```dockerfile
SELECT id, name, price
FROM products
ORDER BY id;
```

Что важно

- SELECT читает строки и не меняет таблицу.
- ORDER BY id обеспечивает стабильный порядок.

Создание товара с RETURNING

```bash
INSERT INTO products (name, price)
VALUES ('Keyboard', 15000)
RETURNING id, name, price;
```

Что важно

- INSERT создаёт запись.
- RETURNING возвращает созданную строку для HTTP-ответа 201.

Изменение и удаление

```
DOCKERFILE
UPDATE products SET price = 12000 WHERE id = 1 RETURNING *;
DELETE FROM products WHERE id = 1 RETURNING *;
```

Что важно

- WHERE ограничивает действие одним товаром.
- Если строка не найдена, RETURNING вернёт пустой результат.

SQL CRUD и маршруты REST API

Таблица products

PostgreSQL
База данных

products

id	name	price
1	Keyboard	15000
2	Mouse	8000
3	Monitor	25000
...	...	...

База данных PostgreSQL

1 CREATE
Создание записи

SQL (PostgreSQL)

```
INSERT INTO products (name, price)
VALUES ('Keyboard', 15000);
```

REST API (Express)

```
POST /api/products
```

2 READ
Чтение данных

SQL (PostgreSQL)

```
SELECT * FROM products;
SELECT * FROM products
WHERE id = 1;
```

REST API (Express)

```
GET /api/products
GET /api/products/:id
```

3 UPDATE
Обновление записи

SQL (PostgreSQL)

```
UPDATE products
SET price = 17000
WHERE id = 1;
```

REST API (Express)

```
PATCH /api/products/:id
```

4 DELETE
Удаление записи

SQL (PostgreSQL)

```
DELETE FROM products
WHERE id = 1;
```

REST API (Express)

```
DELETE /api/products/:id
```

Связь SQL и REST API

Операция	SQL	HTTP	Маршрут
Create (Создание)	`INSERT INTO products (name, price) VALUES (...);`	POST	/api/products
Read (Чтение)	`SELECT * FROM products;` `SELECT * FROM products WHERE id = 1;`	GET	/api/products /api/products/:id
Update (Обновление)	`UPDATE products SET price = 17000 WHERE id = 1;`	PATCH	/api/products/:id
Delete (Удаление)	`DELETE FROM products WHERE id = 1;`	DELETE	/api/products/:id



Клиент
Браузер, Postman или другое приложение

HTTP-запрос
(POST, GET, PATCH, DELETE)

JSON-ответ
(данные или результат)



Express API
Обрабатывает маршрут, выполняет логику и вызывает SQL

SQL-операция
(INSERT, SELECT, UPDATE, DELETE)

Результат из БД
(данные)



PostgreSQL
Выполняет SQL-операцию



Что важно понять

1. REST API получает HTTP-запрос.
2. Express API выполняет логику.
3. PostgreSQL выполняет SQL-операцию.
4. Клиент получает JSON-ответ.

Express, pg и HTTP CRUD

Контроллер получает HTTP, сервис выполняет запрос

Драйвер pg и пул соединений

- Пакет pg обеспечивает связь Node.js с PostgreSQL.
- Pool повторно использует соединения и освобождает код маршрутов от ручного управления каждым соединением.
- Внутри Docker DB_HOST=db, а не localhost: адрес означает другой контейнер.
- Для учебного API достаточно одного пула на процесс Node.js.

config/db.js — создание пула

```javascript
import pg from 'pg';

export const pool = new pg.Pool({
  host: process.env.DB_HOST || 'db',
  port: Number(process.env.DB_PORT || 5432),
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME
});
```

Что важно

- Параметры подключения поступают из окружения.
- Пароль не записывают в исходном коде; учётные значения учебные и не используются в production.

Service: параметризованный SELECT

DOCKERFILE

```
export async function getAll() {
  const result = await pool.query(
    'SELECT id, name, price FROM products ORDER BY id'
  );
  return result.rows;
}
```

Что важно

- pool.query возвращает Promise с rows.
- Список rows будет сериализован Express через res.json().

Service: параметризованный INSERT

BASH

```
export async function create(data) {
  const sql = 'INSERT INTO products (name, price) VALUES ($1, $2) RETURNING *';
  const result = await pool.query(sql, [data.name, data.price]);
  return result.rows[0];
}
```

Что важно

- Параметры $1, $2 передаются отдельно от SQL-текста.
- Нельзя собирать SQL подстановкой пользовательской строки.

Валидация POST и PATCH

- POST принимает JSON-объект с name и price.
- name: строка после trim(), длина 1–100.
- price: число, конечное и неотрицательное; для примера допускаем не более 2 знаков после запятой.
- PATCH принимает name и/или price и меняет только переданные поля.
- Некорректные данные → 400, отсутствующий id → 404.

Controller: GET и POST

```bash
export async function getAll(req, res) {
  const products = await service.getAll();
  res.json(products);
}

export async function create(req, res) {
  const result = await service.create(req.body);
  res.status(201).json(result);
}
```

Что важно

- В полном проекте до create проводится валидация.
- Express 5 передаёт ошибки отклонённых Promise обработчику ошибок.

Router: пять REST-маршрутов

BASH

```
router.get('/', controller.getAll);
router.get('/:id', controller.getById);
router.post('/', controller.create);
router.patch('/:id', controller.update);
router.delete('/:id', controller.remove);
```

Что важно

- Router задаёт метод и URL, но не содержит SQL.
- Контроллер и сервис разделяют HTTP и доступ к данным.

Разделение Router → Controller → Service → PostgreSQL

Клиент

Браузер / Postman

HTTP-запрос

HTTP-запрос

Router

Принимает маршрут

```
GET  /api/products
POST /api/products
```

Controller

Обрабатывает req/res, проверяет входные данные, вызывает Service

Service

Содержит бизнес-логику, формирует SQL-запросы, работает с данными

```
const products = await db_query(
  'SELECT * FROM products'
);
```

PostgreSQL

Хранит данные в таблицах

products

id	name	price	created_at
1	Ноутбук	75000	2024-05-01
2	Смартфон	50000	2024-05-02
...	...	...	...

JSON-ответ

Кто за что отвечает

Router
Связывает URL и HTTP-метод с контроллером

Controller
Получает запрос, валидирует данные, формирует ответ

Service
Выполняет основную логику приложения

PostgreSQL
Сохраняет и возвращает данные

Структура кода (основные файлы)

```
project/
├── routes/        productRoutes.js     // настройка маршрутов
├── controllers/   productController.js // обработка запросов
├── services/      productService.js    // бизнес-логика
└── config/        db.js                // подключение к PostgreSQL
```

Путь запроса (пошагово)

1. Клиент отправляет HTTP-запрос
2. **Router** направляет запрос
3. **Controller** принимает и проверяет данные
4. **Service** выполняет логику и обращается к БД
5. **PostgreSQL** возвращает результат
6. Клиент получает JSON-ответ

Почему NUMERIC может возвращаться строкой

- Драйвер pg по умолчанию передаёт значения типа NUMERIC как строки, чтобы избежать потери точности.
- В нашем API поле price в JSON может выглядеть как "15000.00", даже если при POST передавалось число.
- Не изменяйте глобальный парсер NUMERIC без оценки диапазона и точности; формат цены заранее оговаривайте в контракте API.

Сборка и запуск многоконтейнерного API

Dockerfile, Compose, готовность PostgreSQL и инициализация

Dockerfile для Express API

DOCKERFILE

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

Что важно

- package-lock.json нужен для воспроизводимого npm ci.
- EXPOSE описывает порт контейнера, но не публикует его на хосте.

Как создать таблицу при первом запуске

- SQL-файлы в /docker-entrypoint-initdb.d выполняются официальным образом PostgreSQL при инициализации пустого каталога данных.
- После первого старта с существующим Volume эти файлы не выполняются повторно.
- Изменение SQL-файла само по себе не меняет таблицу в существующей БД.
- Для изменения схемы позже применяют миграции; сейчас используем CREATE TABLE IF NOT EXISTS и отдельную проверку.

Минимальный compose.yaml

```yaml
services:
  nginx:
    image: nginx:stable
    ports:
      - '8080:80'
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - api
  api:
    build: ./api
    environment:
      DB_HOST: db
      DB_PORT: '5432'
      DB_USER: shopuser
      DB_PASSWORD: training_only
      DB_NAME: shop
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:17
    environment:
      POSTGRES_USER: shopuser
      POSTGRES_PASSWORD: training_only
      POSTGRES_DB: shop
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/01-init.sql:ro
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U shopuser -d shop']
      interval: 5s
      timeout: 3s
      retries: 10
volumes:
  pgdata:
```

Что важно

- Compose создаёт сеть по умолчанию: имена nginx, api, db доступны как DNS-имена.
- Это учебный пример с демонстрационным паролем; реальные секреты нельзя помещать в Git.

Nginx — проксирование API

```bash
server {
  listen 80;
  location / {
    proxy_pass http://api:3000;
  }
}
```

Что важно

- Из хоста открывают localhost:8080.
- Между Nginx и Express используется имя сервиса api и внутренний порт 3000.

1. Путь запроса Nginx → Express API → PostgreSQL



2. SQL CRUD и маршруты REST API

Операция	HTTP метод	Маршрут (Endpoint)	Пример запроса	SQL операция

CREATE (Создание)	POST	/api/products	POST /api/products
			Content-Type: application/json
			{
			"name": "Ноутбук",
			"price": 50000
			}
				INSERT INTO products
				(name, price)
				VALUES ('Ноутбук', 50000);
				-- возвращает id новой записи

READ (Чтение всех)	GET	/api/products	GET /api/products	SELECT * FROM products;
READ (Чтение по ID)	GET	/api/products/1	GET /api/products/1	SELECT * FROM products WHERE id = 1;

UPDATE (Изменение)	PATCH	/api/products/1	PATCH /api/products/1
			Content-Type: application/json
			{
			"price": 55000
			}
				UPDATE products
				SET price = 55000
				WHERE id = 1;

DELETE (Удаление)	DELETE	/api/products/1	DELETE /api/products/1	DELETE FROM products WHERE id = 1;

3. Разделение Router → Controller → Service → PostgreSQL

Routes (Маршруты) — productRoutes.js

```
const router = require('express').Router();
const controller = require('../controllers
    /productController');

router.get('/', controller.getAll);
router.get('/:id', controller.getById);
router.post('/', controller.create);
router.patch('/:id', controller.update);
router.delete('/:id', controller.remove);

module.exports = router;
```

Controller (Контроллер) — productController.js

```
const service = require(
  '../services/productService');

exports.getAll = async (req, res) => {
  const products = await service.getAll();
  res.json(products);
};

exports.create = async (req, res) => {
  const product = await service.create(
  req.body);
  res.status(201).json(product);
};

// ... другие методы
```

Service (Бизнес-логика) — productService.js

```
const db = require('../config/db');

exports.getAll = async () => {
  const { rows } = await db.query(
  'SELECT * FROM products
   ORDER BY id');
  return rows;
};

exports.create = async (data) => {
  const { name, price } = data;
  const { rows } = await db.query(
  'INSERT INTO products (name, price)
   VALUES ($1, $2) RETRUNING *',
   [name, price]
  )
  return rows[0];
};

// ... другие метрды
```

PostgreSQL (База данных)

products

id	SERIAL
name	VARCHAR(100)
price	NUMERIC(10,2)

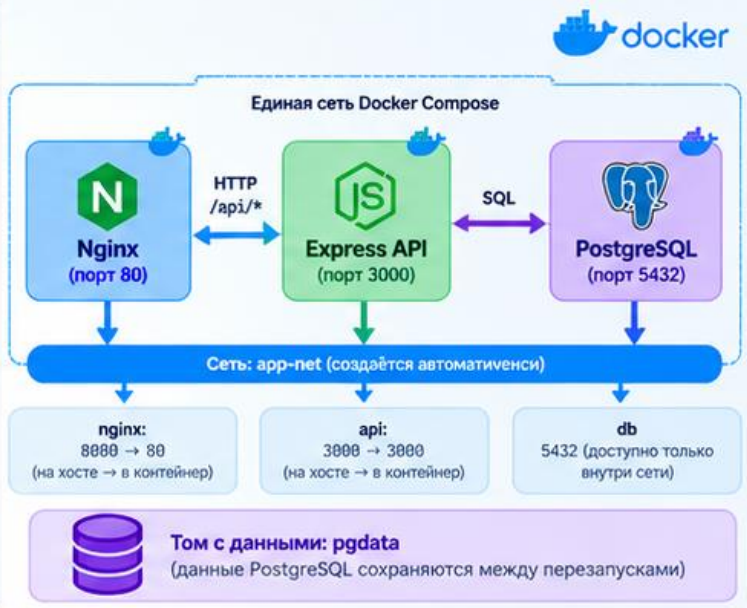
```
SELECT * FROM products;
INSERT INTO products ...
UPDATE products ...
DELETE FROM products ...
```

4. Docker Compose с REST API, Nginx и PostgreSQL

docker-compose.yaml

```
version: '3.9'
pestyres:
  nginx:
    image: nginx:latest
    ports:
      - "80:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.ddefault.conf
    depends_on:
      - api
  api:
    build: ./api
    ports:
      - "3000:3000"
    environment:
      DB_HOST: db
      DD_PORT: 5432
    depends_on:
      - db
  db:
    image: postgres:17
    environment:
      POSTGRES_DB: shop
      POSTGRES_USER: shopuser
      POSTGRES_PASSWORD: shoppiass
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```



Первый запуск проекта

1	**Шаг 1**	1. Проверить compose.yaml командой docker compose config.
2	**Шаг 2**	2. Собрать API и запустить стек: docker compose up -d --build.
3	**Шаг 3**	3. Проверить docker compose ps и готовность db.
4	**Шаг 4**	4. Выполнить GET /api/products через опубликованный порт Nginx.
5	**Шаг 5**	5. Выполнить POST, PATCH и DELETE через curl.
6	**Шаг 6**	6. Проверить SQL-таблицу и сохранность данных после down/up.

Проверка, диагностика и устойчивость

Как подтвердить, что работает весь путь запроса

Проверка через Nginx

```bash
curl -i http://127.0.0.1:8080/api/products
curl -i -X POST http://127.0.0.1:8080/api/products \
  -H 'Content-Type: application/json' \
  -d '{"name":"SSD","price":42000}'
```

Что важно

- GET должен вернуть 200 и JSON-массив.
- POST при правильном теле возвращает 201 и новый id.

Диагностика Compose

DOCKERFILE

```
docker compose ps
docker compose logs --tail=30 api
docker compose logs --tail=30 db
docker compose exec db psql -U shopuser -d shop -c 'SELECT * FROM products;'
```

Что важно

- ps показывает состояние контейнеров.
- logs показывает ошибки приложения и БД.
- psql проверяет фактические записи независимо от HTTP API.

HTTP-коды и типовые неисправности

Ситуация	HTTP/симптом	Проверка
Список товаров	200	GET /api/products
Создание товара	201	POST /api/products
Ошибка входных данных	400	JSON и валидация
Товар отсутствует	404	Проверка id
API недоступен	502 от Nginx	docker compose logs nginx api
База недоступна	5xx	docker compose ps; logs db

Диагностический эксперимент

Шаг 1	1. Остановить db: docker compose stop db.
Шаг 2	2. Запросить GET /api/products и зафиксировать ошибку.
Шаг 3	3. Проверить docker compose ps и logs api/db.
Шаг 4	4. Восстановить: docker compose start db.
Шаг 5	5. Дождаться готовности, повторить запрос и убедиться в успешном результате.

Сохранность данных и предосторожности

- docker compose down удаляет контейнеры и сеть проекта, но сохраняет именованный том.
- docker compose down -v удаляет том проекта: данные БД могут быть потеряны.
- Для существующей БД SQL-скрипты из /docker-entrypoint-initdb.d не запускаются повторно.
- Volume полезен для постоянных данных, но не заменяет резервное копирование.

Итоги лекции

1. Наружу публикуется nginx; api и db доступны внутри Compose-сети.
2. Express обрабатывает HTTP, PostgreSQL хранит данные; SQL выполняется через pg.Pool.
3. CRUD реализуется отдельными Router, Controller и Service.
4. Параметризованные запросы защищают от SQL-инъекций.
5. Healthcheck помогает дождаться готовности БД.
6. Данные PostgreSQL сохраняются в именованном Docker Volume.

Контрольные вопросы

1. 1. Почему API подключается к db, а не к localhost?
2. 2. Что делает Nginx в многоконтейнерном приложении?
3. 3. Чем POST отличается от PATCH?
4. 4. Зачем SQL-параметры $1, $2?
5. 5. Для чего нужен Pool в pg?
6. 6. Почему NUMERIC может вернуться строкой в JSON?
7. 7. Когда выполняется init.sql из docker-entrypoint-initdb.d?
8. 8. Чем отличаются docker compose down и down -v?
9. 9. Для чего healthcheck и service_healthy?
10. 10. Как проверить данные без обращения к REST API?